Writer Monad

Writer · Compute totals while accumulating audit log · Build with Waqar

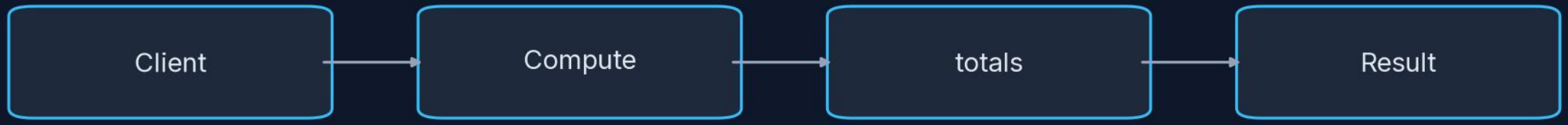
Today's Hook

- Today's force: Compute totals while accumulating audit log
- We earn Writer by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: Writer, Audited

The Problem

- Scenario: Compute totals while accumulating audit log
- Constraints: solve «Compute totals while accumulating audit log» with Writer — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Compute totals while accumulating audit log
- Success looks like: 30

Structure



Writer: Compute totals while accumulating audit log

Real-World Scenario

- Running example: Compute totals while accumulating audit log
- Key types: Writer, Audited
- Happy-path stdout cue: 30
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```java
package com.buildwithwaqar.patterns.functional.writer.problem;

import java.util.*;

public final class NaiveAudit {
  public static int total(List<Integer> xs){ return xs.stream().mapToInt(i->i).sum(); }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Compute totals while accumulating audit log
- Without Writer, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```
package com.buildwithwaqar.patterns.functional.writer.problem;

import java.util.*;
import java.util.function.*;

record Writer<W,A>(A value, List<W> log){
  static <W,A> Writer<W,A> pure(A a){ return new Writer<>(a, List.of()); }
  <B> Writer<W,B> map(Function<A,B> f){ return new Writer<>(f.apply(value), log); }
  <B> Writer<W,B> flatMap(Function<A, Writer<W,B>> f){
    var next=f.apply(value); List<W> merged=new ArrayList<>(log); merged.addAll(next.log); return new Writer<>(next.value, merged);
  }
  static <W,A> Writer<W,A> tell(W w, A a){ return new Writer<>(a, List.of(w)); }
}
final class Audited {
  static Writer<String,Integer> addLine(int cents){ return Writer.tell("line="+cents, cents); }
  static Writer<String,Integer> total(List<Integer> lines){
    Writer<String,Integer> acc=Writer.pure(0);
    for(int c: lines){ acc = acc.flatMap(sum -> addLine(c).map(v -> sum+v)); }
    return acc;
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.writer.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Writer: totals with audit log ===");
    System.out.println("--- naive ---"); System.out.println(NaiveAudit.total(List.of(10,20)));
System.out.println("--- writer ---");
var w=Audited.total(List.of(10,20,30));
System.out.println("value="+w.value()); System.out.println("log="+w.log());
  }
}
```

Walkthrough Output

```
=== Writer: totals with audit log ===
--- naive ---
30
--- writer ---
value=60
log=[line=10, line=20, line=30]
```

Step-by-Step Trace

- Match stdout to stages — look for: 30
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp … 55-writer ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```java
package com.buildwithwaqar.patterns.functional.writer.problem;

import java.util.*;
import java.util.function.*;

record Writer<W,A>(A value, List<W> log){
  static <W,A> Writer<W,A> pure(A a){ return new Writer<>(a, List.of()); }
  <B> Writer<W,B> map(Function<A,B> f){ return new Writer<>(f.apply(value), log); }
  <B> Writer<W,B> flatMap(Function<A, Writer<W,B>> f){
    var next=f.apply(value); List<W> merged=new ArrayList<>(log); merged.addAll(next.log); return new Writer<>(next.value, merged);
  }
  static <W,A> Writer<W,A> tell(W w, A a){ return new Writer<>(a, List.of(w)); }
}
final class Audited {
  static Writer<String,Integer> addLine(int cents){ return Writer.tell("line="+cents, cents); }
  static Writer<String,Integer> total(List<Integer> lines){
    Writer<String,Integer> acc=Writer.pure(0);
    for(int c: lines){ acc = acc.flatMap(sum -> addLine(c).map(v -> sum+v)); }
    return acc;
  }
}
```

Trade-offs

- Use Writer when «Compute totals while accumulating audit log» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Writer in product language.

Related Patterns

- Name the pattern usually confused with Writer and separate the forces.
- Sketch the roles for «Compute totals while accumulating audit log» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Compute totals while accumulating audit log
- Naive pain → Writer boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.